

Series de tiempo

Natalia Acevedo Prins.

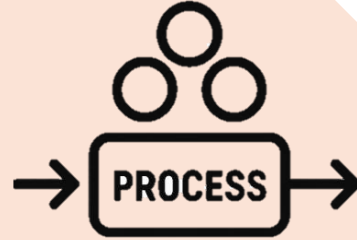
nmacevedop@unal.edu.co

Universidad Nacional de Colombia

2025

Procesos deterministas vs. estocásticos

Proceso determinístico:



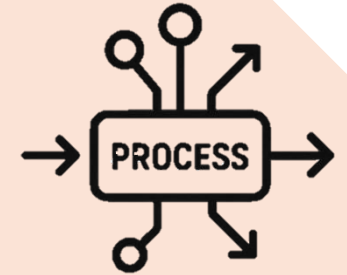
Es aquel en el que los valores futuros de la variable están completamente determinados por una relación funcional o una regla matemática **sin intervención del azar**.

- ✓ Si se conocen las condiciones iniciales y las ecuaciones del modelo, se puede **predecir exactamente** el valor futuro.
- ✓ No existe variabilidad aleatoria: el resultado es **único** y reproducible.

Aplicación típica:

Modelos físicos o de ingeniería donde las leyes son exactas

Proceso estocástico:



Es un proceso donde los valores futuros **dependen del azar**, están sujetos a **variaciones aleatorias** que no pueden predecirse con exactitud.

- ✓ Se modela como una **colección de variables aleatorias** indexadas en el tiempo.
- ✓ Se describe el comportamiento **probabilístico** del sistema.
- ✓ Importancia de la **distribución** de probabilidades y la **dependencia temporal**.

Aplicación típica:

Modelos donde siempre hay incertidumbre en la evolución de la variable.

Definición de Serie de Tiempo

Secuencia de observaciones indexadas por el tiempo ,
donde y_t representa el conjunto de índices temporales



Predicciones climáticas



Análisis de cotizaciones bursátiles



Seguimiento de variables demográficas

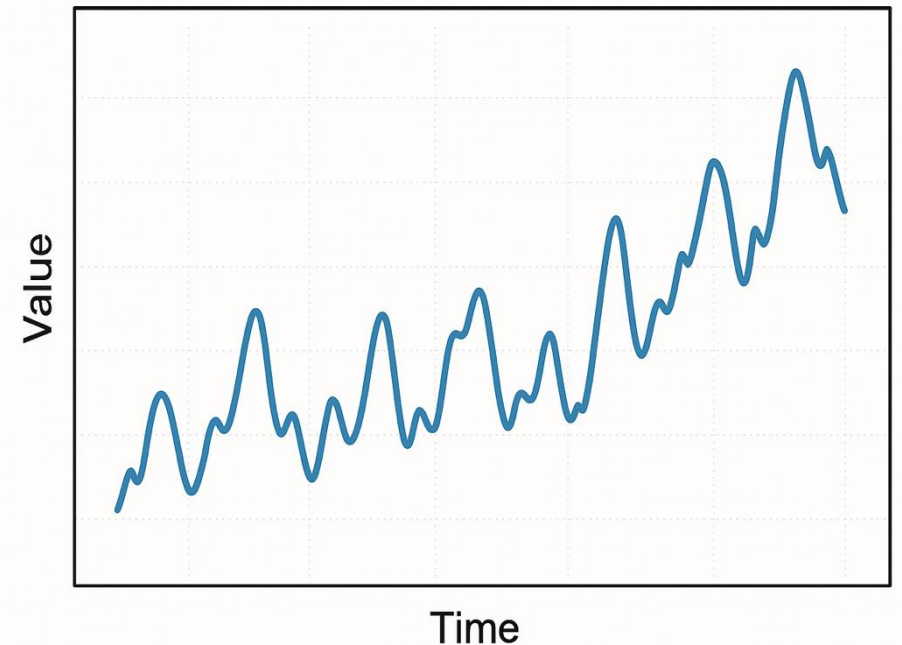


Estudios económicos



mediciones de procesos industriales

El objetivo principal del análisis de series temporales es modelar la dependencia temporal entre las observaciones, es decir, para diferentes valores de rezago .



Metodología para abordar un problema de series de tiempo

Enfoque Box-Jenkins

Definición del objetivo

¿Qué buscas?

- ✓ ¿Pronosticar valores futuros?
- ✓ ¿Entender la dinámica interna (autocorrelación, estacionalidad, shocks)?
- ✓ ¿Te interesa el comportamiento medio, la varianza?

Revisión y Limpieza

Verificar:

- ✓ Frecuencia.
- ✓ Periodo cubierto (Ventana de observación).
- ✓ Unidades de medida.
- ✓ Presencia de valores.
- ✓ faltantes o duplicados.
- ✓ Cambios estructurales .

Visualización y exploración

Exploración gráfica:

- ✓ Serie original (tendencia, estacionalidad, cambios estructurales).
- ✓ Zoom por años o meses.
- ✓ Diferentes escalas o transformaciones.

Estacionariedad y Transformaciones

Verificar:

- ✓ Gráficos: media y varianza a lo largo del tiempo.
- ✓ Test formales:
 - ✓ Dickey–Fuller aumentado (ADF)
 - ✓ KPSS
 - ✓ Phillips–Perron

Elección del modelo

Diagnóstico de Residuos

Pronóstico y Evaluación

Reportes

Propiedades de las series de tiempo

Tendencia

Movimiento de largo plazo (creciente o decreciente) que refleja la evolución general de la serie.

Estacionalidad

Variaciones periódicas de corto plazo asociadas al calendario (meses, estaciones, días).

Variación cíclica

Oscilaciones de largo plazo relacionadas con ciclos económicos o sociales de duración variable.

Ruido aleatorio

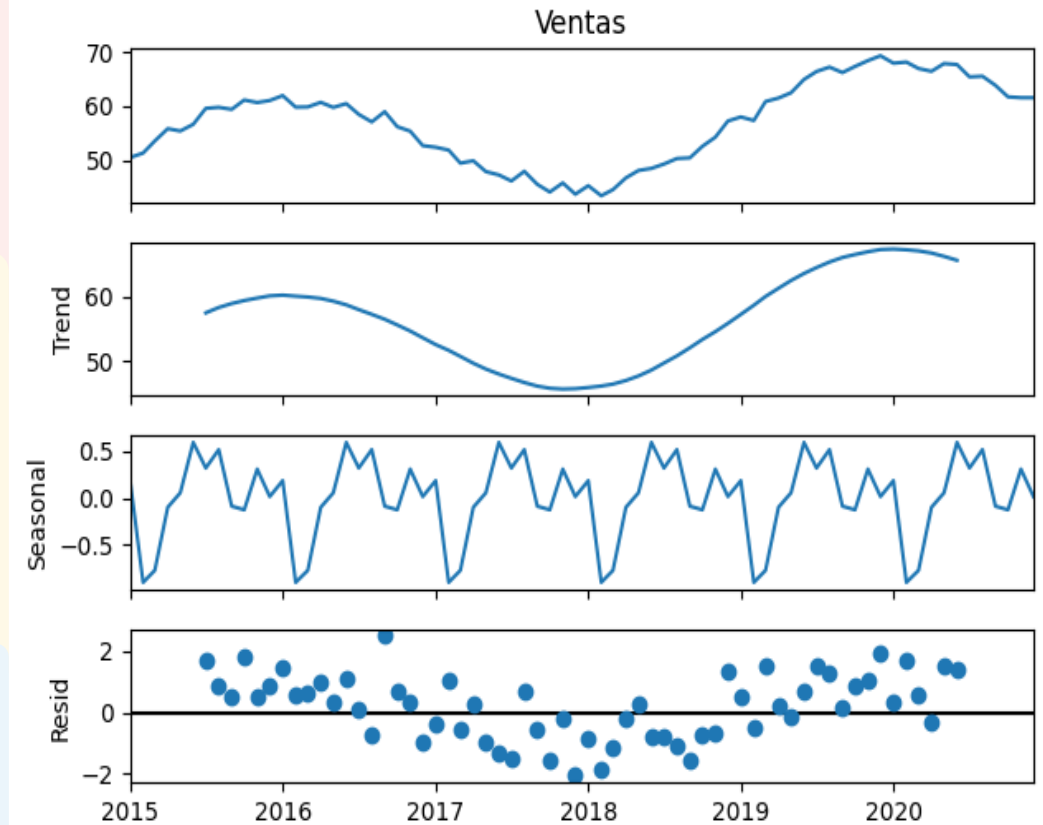
Componente impredecible ocasionado por eventos azarosos, errores de medición u otras irregularidades.

Variación transitoria

Fenómenos aislados que alteran temporalmente el comportamiento (cambios estructurales).

Estacionariedad

Propiedad de la serie donde la media y la varianza son constantes; no hay tendencia ni estacionalidad.



Descomposición de las series

La descomposición separa la serie observada Y_t en componentes interpretable(s):

- ✓ Tendencia/ciclo T_t
- ✓ Estacionalidad S_t :patrón repetitivo con periodo s
- ✓ Resto R_t :ruido y eventos idiosincráticos

Formas del modelo:

Aditivo:

$$Y_t = T_t + S_t + R_t$$

Multiplicativo:

$$Y_t = T_t \times S_t \times R_t$$

Equivalente aditivo en log:

$$\log Y_t = \log T_t + \log S_t + \log R_t.$$

Elección rápida: si la estacionalidad aumenta con el nivel, use multiplicativo

Paso 0. Preparación

Paso 1. Estimar la tendencia T_t

Paso 2. Quitar tendencia para aislar estacionalidad

Paso 3. Estimar estacionalidad S_t

Paso 4. Obtener el resto R_t

Paso 5. Diagnóstico

Paso 6. Reconstrucción
 $\hat{Y}_t = \hat{T}_t + \hat{S}_t(+) \hat{R}_t.$

Concepto de raíz unitaria

La **raíz unitaria** es el concepto central para distinguir series no estacionarias con tendencia estocástica de las estacionarias.

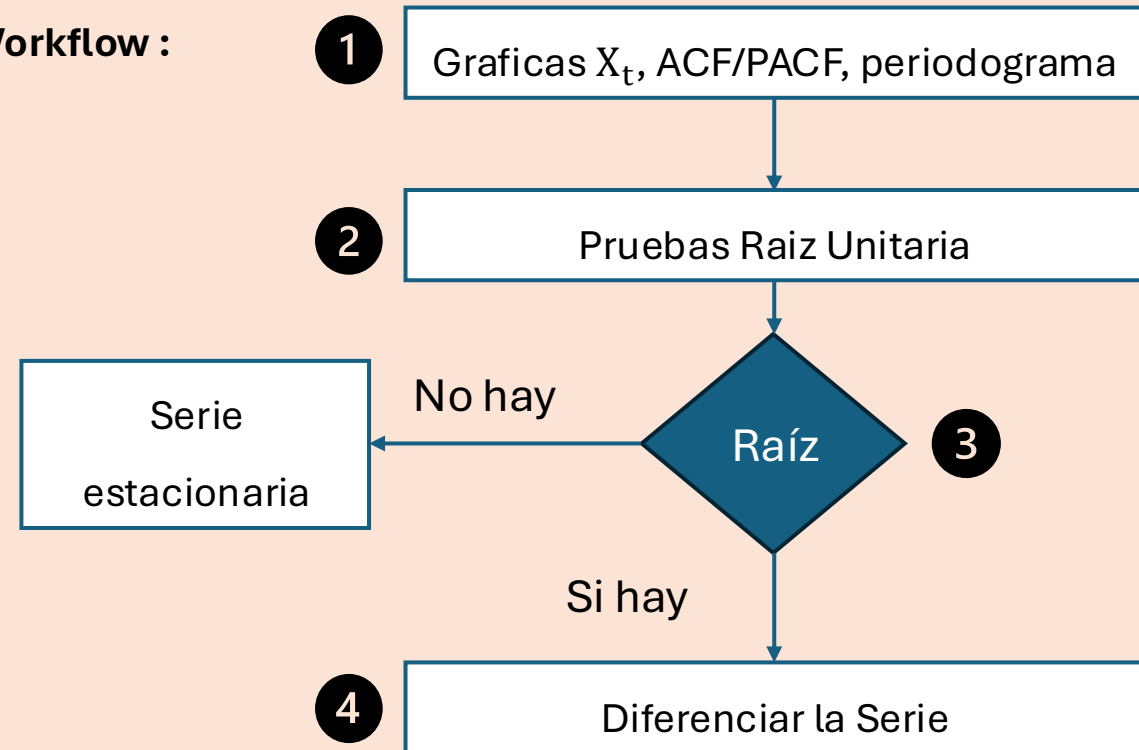
Pruebas de Raíz Unitaria:

- ✓ Regresión Dickey–Fuller ampliada (ADF)
- ✓ KPSS: nulo de estacionariedad (lo opuesto a ADF)
- ✓ Phillips–Perron (PP)

Consecuencias de tener raíz unitaria:

- ✓ **No estacionariedad:** media y varianza dependen de t .
- ✓ **Persistencia permanente:** los choques no se disipan; la respuesta al impulso no es sumable.
- ✓ **Regresiones espurias:** Resultados sin significancia estadística.

Workflow :



Transformaciones y Diferenciación

Transformaciones



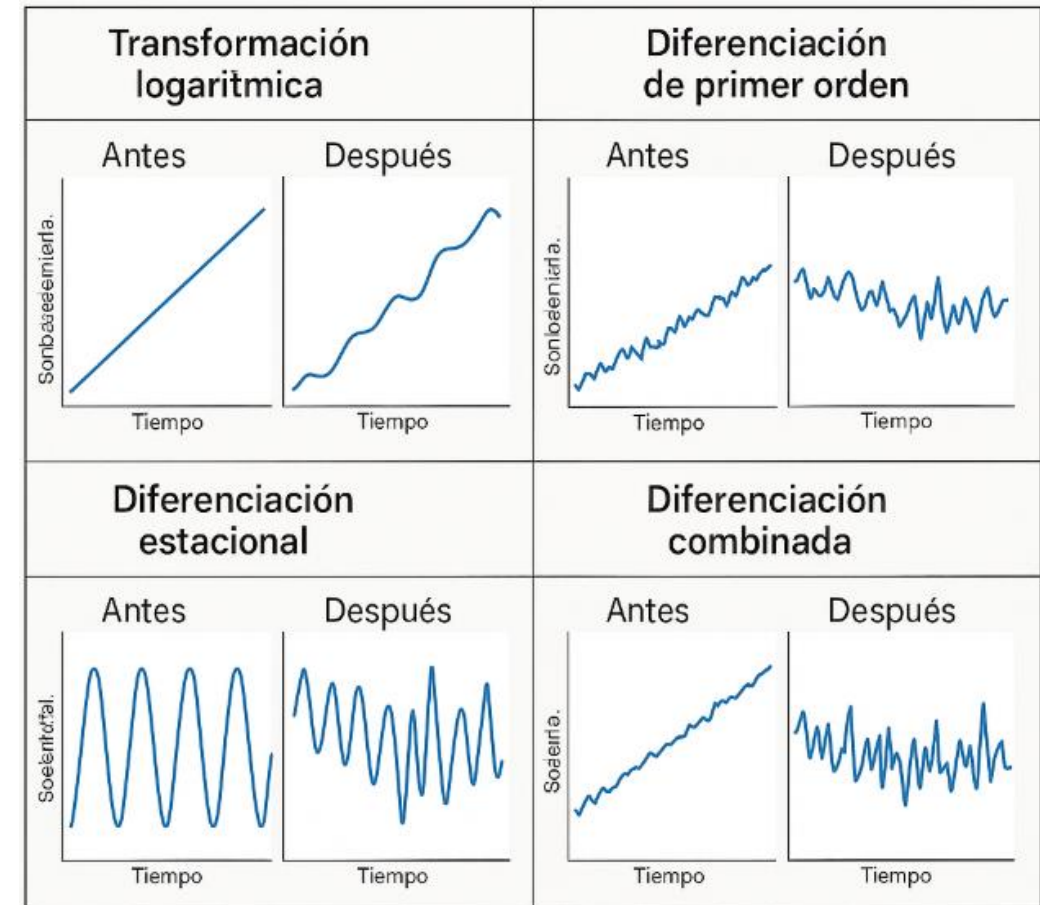
Estabilizar la varianza

Diferenciaciones



Eliminar tendencias y raíces unitarias

Problema	Síntoma	Transformación recomendada
Varianza creciente	Aumentos proporcionales	Log o Box-Cox
Tendencia lineal	ACF decae lentamente	Diferencia de primer orden
Tendencia cuadrática	ACF muy persistente	Diferencia de segundo orden
Estacionalidad	Picos ACF en múltiplos de s	Diferencia estacional
Cambios de nivel abruptos	Media no constante	Diferencia regular
Volatilidad agrupada	Varianza condicional variable	Modelos GARCH (no solo transformación)



Autocorrelación

Medida estadística que indica el **grado de relación entre los valores de una serie de tiempo y sus propios valores pasados.**

Alta o
baja?

Alta

Valores actuales dependen fuertemente de los anteriores.

Cercana a cero

Los valores son prácticamente independientes.

Herramientas para Identificar Autocorrelación:

1. Función de Autocorrelación (ACF)
2. Función de Autocorrelación Parcial (PACF)
3. Pruebas estadísticas
4. Inspección visual

ACF:

Mide la **correlación lineal** entre los valores de la serie y sus rezagos temporales

$$\rho_k = \frac{\sum_{t=k+1}^n (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^n (y_t - \bar{y})^2}$$

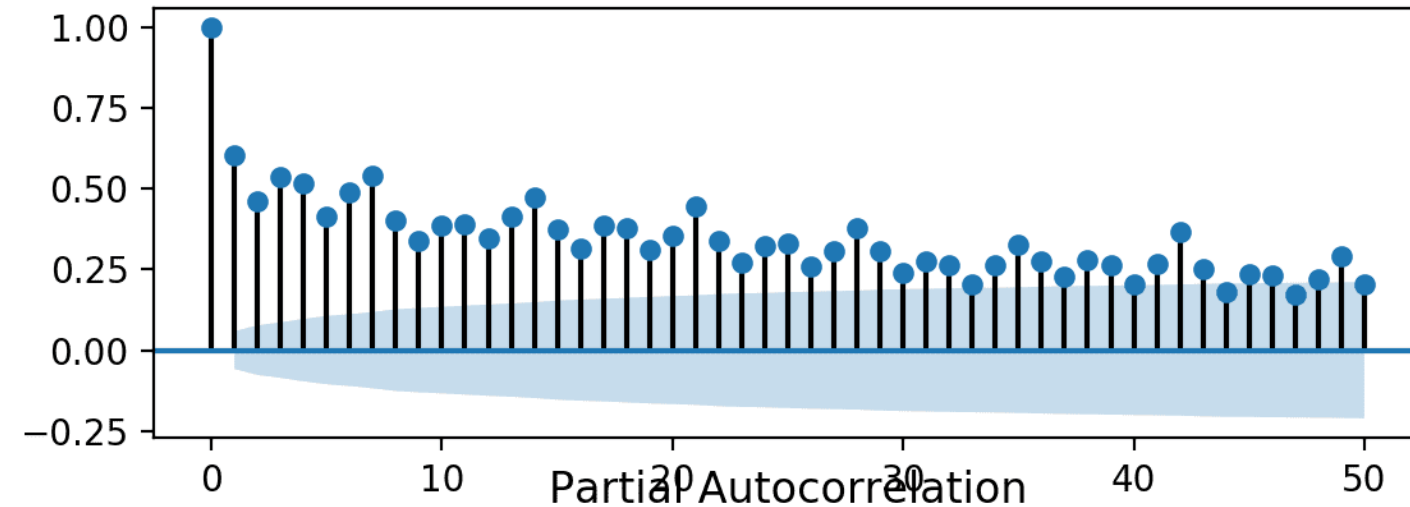
PACF:

Mide la **correlación lineal** entre los valores de la serie y sus rezagos temporales

$$\phi_{kk} = \text{corr}(y_t' - \hat{y}_t, y_{t-k} - \widehat{y_{t-k}})$$

ACF y PACF

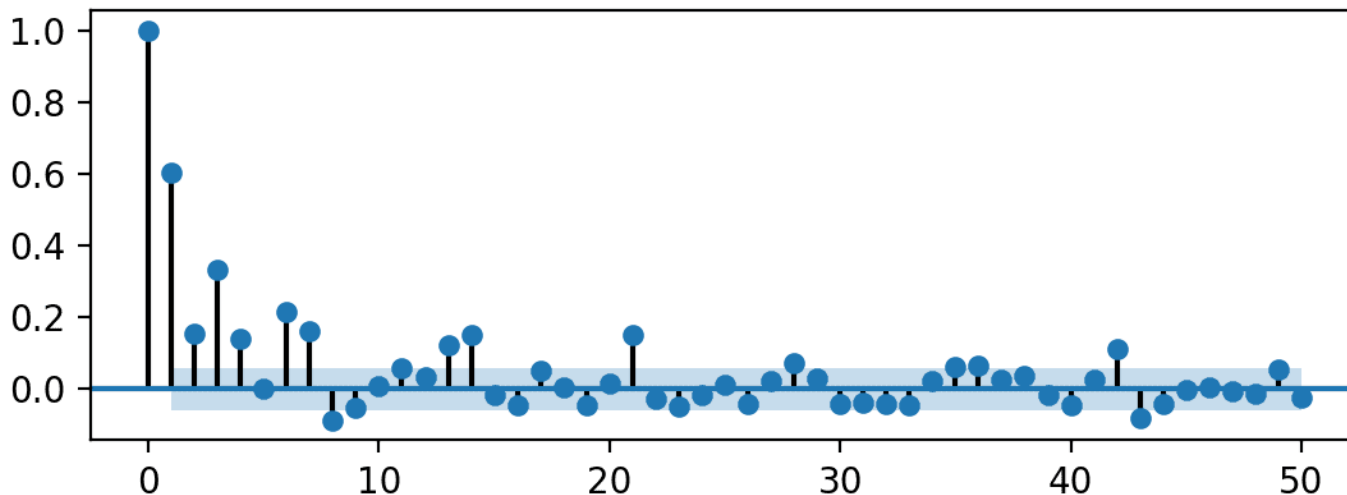
Autocorrelation



Interpretar el patrón:

- ✓ Si varias barras están fuera del intervalo → existe autocorrelación significativa.
- ✓ Si decae lentamente → la serie no es estacionaria.
- ✓ Si cae rápidamente a cero → la serie es estacionaria.

Partial Autocorrelation



Interpretar el patrón:

- ✓ Un corte brusco (spike) en el rezago p sugiere un modelo $AR(p)$.
- ✓ Un decaimiento gradual sugiere un modelo $MA(q)$ o mezcla ARMA.

Ruido Blanco

Una serie $\{\varepsilon_t\}$ es ruido blanco si :

$$\mathbb{E}[\varepsilon_t] = 0$$

$$\text{Var}(\varepsilon_t) = \sigma^2 < \infty$$

$$\text{Cov}(\varepsilon_t, \varepsilon_{t-h}) = 0$$

- ✓ En series financieras, los residuos suelen mostrar volatilidad agrupada (heterocedasticidad).
- ✓ Aunque la media siga un modelo ARMA, los residuos no son ruido blanco gaussiano, sino ruido blanco con varianza condicional (base de los modelos ARCH/GARCH).

Ruido blanco fuerte Si $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$ i.i.d., es ruido blanco gaussiano.

En los modelos clásicos de series de tiempo (AR, MA, ARMA, ARIMA, SARIMA, etc.), se espera que los residuos sean un ruido blanco gaussiano.

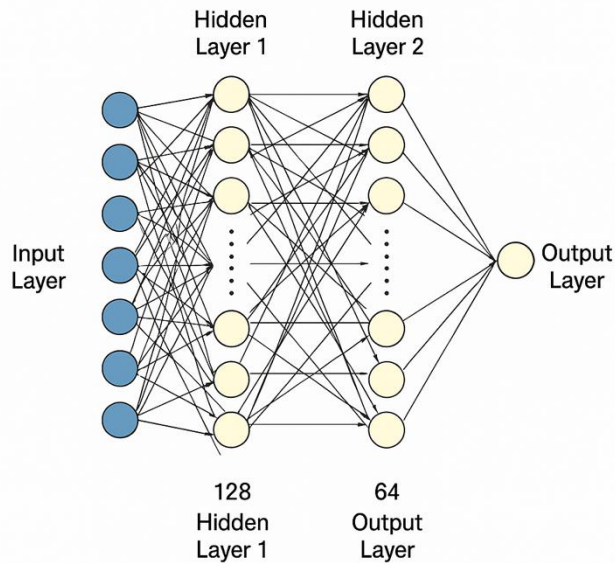
Residuales de una Serie con Ruido Blanco



Redes Neuronales para Series de Tiempo

Redes Neuronales: Perceptrón Multicapa-MLP

Un **modelo MLP (Perceptrón Multicapa)** es una red de “neuronas artificiales” que aprende a reconocer patrones en los datos.



$$x_i \rightarrow f\left(\sum_{i=1}^n w_i x_i + b\right) \rightarrow y_i$$

x_i = valores de entrada

w_i = pesos que indican la importancia de cada entrada

b = sesgo (bias), que ajusta la salida

$f(\cdot)$ = función de activación,

y = salida del nodo.

Cuando se **conectan varias capas**, cada capa toma la salida de la anterior y la transforma antes de pasarla a la siguiente.



Las primeras capas detectan patrones simples (por ejemplo, tendencias o formas básicas).



Las capas intermedias combinan esos patrones.



Las últimas capas aprenden relaciones más abstractas y generan la **predicción final**.

Redes Neuronales: Perceptrón Multicapa-MLP



1. Planteamiento del problema

1. **Variable:** serie y_t con frecuencia fija.
2. **Horizonte**
3. **Estrategia de pronóstico**
 - ✓ **Recursiva:** entrenas $h = 1$ y realimentas predicciones.
 - ✓ **Directa:** un modelo por cada h .
 - ✓ **Multi-salida:** un solo modelo que predice $(\hat{y}_{t+1}, \dots, \hat{y}_{t+H})$.



2. Preparación y saneamiento de datos

- ✓ Alineación y frecuencia
- ✓ Faltantes
- ✓ Outliers
- ✓ Transformaciones



3. Ingeniería de características sin fuga

- ✓ **Deslizador para MLP:** el MLP espera un vector fijo $x_t = [y_{t-1}, \dots, y_{t-p}]$.
- ✓ **Lags:** elige p con ACF/PACF como prior y confirma en validación.



4. Partición temporal (sin fuga)

- ✓ **Entrenamiento** → **Validación** → **Test**



5. Definición del modelo

- ✓ **Arquitectura:** 1–3 capas densas con activación ReLU o GELU sigmoide
- ✓ **Salida:** 1 neurona por horizonte (directa) o H neuronas (multi-salida).
- ✓ **Regularización:** dropout moderado, L2 (weight decay), early stopping.
- ✓ **Optimización:** Adam/AdamW; *batch size* moderado; *learning rate* inicial pequeño con *decay*.



6. Función de pérdida y métricas

- ✓ **Pérdida de entrenamiento:** RMSE/MSE .
- ✓ **Métricas de evaluación**



7. Validación y tuning (walk-forward)

- ✓ Búsqueda de hiperparámetros.
- ✓ Validación Cruzada



8. Entrenamiento final y pronóstico

- ✓ Re-entrenar el MLP ganador con data completa.
- ✓ Generar pronósticos
- ✓ Deshacer transformaciones



9. Puesta en producción

- ✓ Empaqueta pipeline completo (imputación → escalado → construcción de ventana → MLP → inversas).

Preprocesamiento sin fugas (Feature Engineering)

La fuga de información (data leakage) ocurre cuando el modelo accede (directa o indirectamente) a **información del futuro** al momento del entrenamiento.



El modelo “aprende” con datos que en el momento de la predicción **no existirían aún**.

En práctica:

$$x_t = f(y_{t-1}, y_{t-2}, y_{t-3}, \dots)$$

No puede depender de y_t ni de ningún y_{t+h} con $h > 0$

Tipo	Ejemplo	Problema
Rolling centrado	<code>y.rolling(7, center=True).mean()</code>	❌ usa valores futuros
Diferencias futuras	<code>y.diff(-1)</code>	❌ accede a (y_{t+1})
Media global de todo el dataset	<code>mean(y)</code>	❌ incluye observaciones futuras
Normalización con toda la serie	<code>scaler.fit(X_full)</code> antes del split	❌ parámetros del scaler usan información futura

Cómo evitar la fuga en la práctica (checklist):

- ✓ Aplica `fit()` de *scalers* o *imputers* **solo con el conjunto de entrenamiento**
- ✓ Siempre aplica `.shift(1)` antes de `.rolling()`
- ✓ Evita funciones con `center=True` o que miren hacia adelante.
- ✓ En **validación cruzada temporal**: Nunca mezcles o barajes (no hay `shuffle=True`).



Preprocesamiento: Escalado

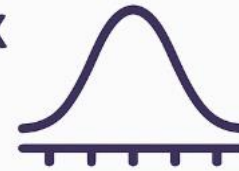


Los **métodos de escalado** son técnicas que transforman los datos para que todos sus valores estén dentro de un mismo rango.

Escalar las **series de tiempo** es importante porque:

- ✓ Evita que las variables con valores grandes dominen el aprendizaje.
- ✓ Mejora la **convergencia** del modelo al entrenarse más rápido.
- ✓ Aumenta la **precisión** y **estabilidad numérica** de los algoritmos de *machine learning*.

MIN MAX

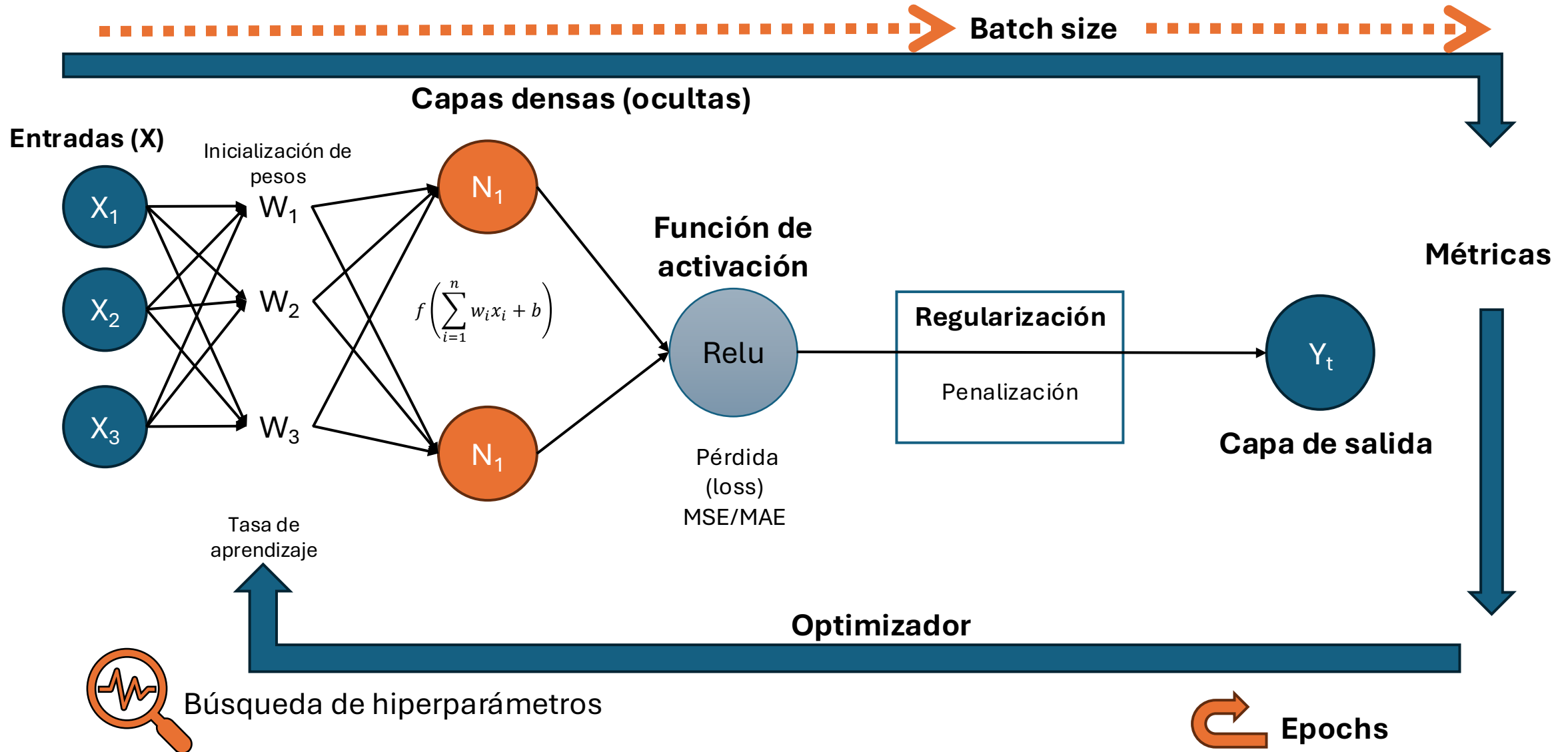


`sklearn.preprocessing`

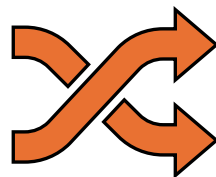
Escalador	Descripción	Rango típico	Cuándo usarlo
MinMaxScaler	Escala los datos a un rango fijo.	[0, 1] (por defecto)	Cuando se requiere mantener la forma de la serie y limitar los valores.
StandardScaler	Centra los datos con media 0 y desviación estándar 1.	Depende de los datos	Cuando se usan modelos sensibles a la varianza (redes neuronales, SVM).
RobustScaler	Usa la mediana y el rango intercuartílico (IQR), resistente a valores atípicos.	Depende de los datos	Ideal cuando existen outliers en la serie.
QuantileTransformer	Transforma los datos según su distribución acumulada (CDF).	[0, 1] o distribución normal	Para obtener una distribución más uniforme o normalizada.

Construcción del modelo

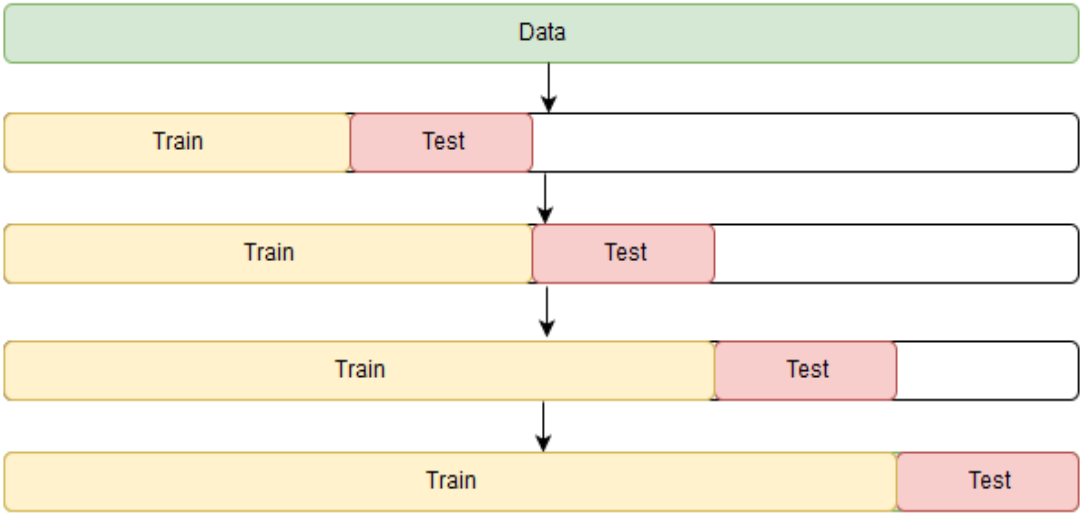
Multi-Layer Perceptron MLP



Cross Validation



El **cross validation** es un proceso que consiste en **dividir los datos en varios grupos (o “folds”)** para entrenar el modelo en una parte y probarlo en otra, rotando las divisiones.



Sirve para **evaluar el rendimiento real del modelo** y evitar el sobreajuste, asegurando que funcione bien con datos nuevos.



No es bootstrapping

Aspecto	Cross Validation	Bootstrapping
Forma de muestreo	Divide los datos en k partes sin reemplazo.	Toma muestras con reemplazo del conjunto original.
Uso de los datos	Cada observación se usa una vez para prueba y varias veces para entrenamiento.	Algunas observaciones pueden repetirse, otras pueden no aparecer.
Objetivo principal	Evaluar el rendimiento del modelo de forma estable.	Estimar la variabilidad o incertidumbre de un estimador o métrica.
Ventaja	Más representativo del rendimiento general.	Permite calcular intervalos de confianza y estabilidad del modelo.
Desventaja	Más costoso computacionalmente si k es grande.	Puede introducir más sesgo por la repetición de datos.

LSTM para Series de tiempo

Es un tipo de **red neuronal recurrente** capaz de aprender y recordar **dependencias a largo plazo** en datos secuenciales mediante un **sistema de compuertas** que controla qué información se almacena, se actualiza o se olvida en cada paso temporal.



Poseen memoria interna controlada por compuertas

Aprenden la dinámica temporal directamente de los datos



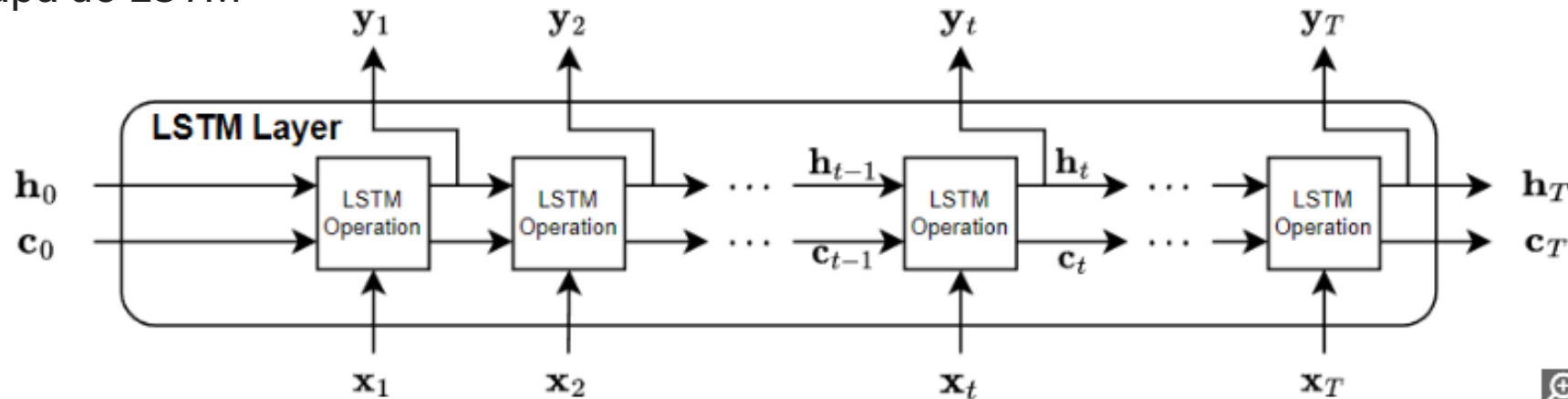
Aprenden dependencias a largo plazo

Capacidad para modelar relaciones no lineales complejas

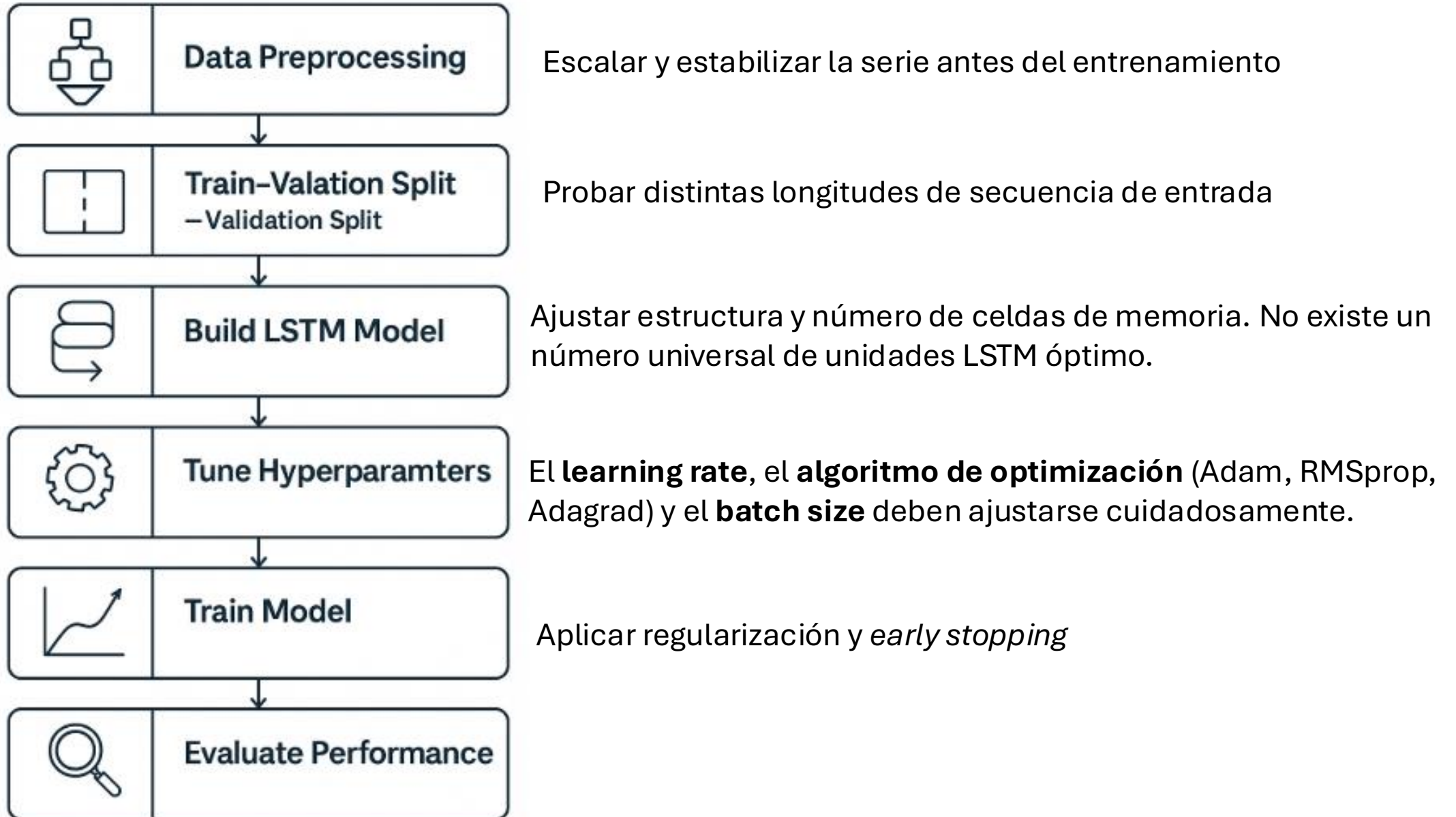
No requieren supuestos fuertes sobre la estructura del modelo.



Flujo de datos a través de una capa de LSTM



Metodología



Problemas frecuentemente hallados

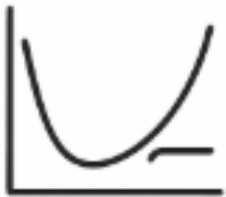


Sobreajuste (Overfitting)

El modelo aprende **demasiado bien el conjunto de entrenamiento**, capturando ruido o patrones espurios.

Subajuste (Underfitting)

El modelo no logra capturar ni siquiera los patrones del entrenamiento.



Gradientes que se desvanecen (Vanishing gradients)

Durante la retropropagación, los gradientes se vuelven muy pequeños → las capas iniciales no aprenden.

Gradientes explosivos (Exploding gradients)

Los gradientes crecen de forma descontrolada → pesos enormes → divergencia numérica.



Mínimos locales y puntos de ensilladura

El optimizador se queda atrapado en regiones del espacio de pérdida con gradientes casi nulos pero no óptimos.

Problemas frecuentemente hallados



Mala elección de tasa de aprendizaje

El learning rate (LR) controla el tamaño del paso en el descenso de gradiente.

Muy alto → diverge.

Muy bajo → no converge o se estanca.

Datos mal preparados o escalados

Si los datos no están centrados ni escalados, los gradientes y activaciones pueden saturarse

Datos insuficientes o poco informativos

Si la muestra no tiene suficiente variabilidad o no refleja la dinámica real, el modelo no puede aprender patrones útiles.



Generalización débil (fuga de información)

El modelo “ve” información del futuro en el entrenamiento (muy común en series temporales).

Soluciones

Síntoma observado	Soluciones recomendadas
 Pérdida baja en train pero alta en val	- Aumentar Dropout o L2. - Implementar EarlyStopping(patience=10). - Reducir número de neuronas/capas.- Aumentar datos (más historia o ruido leve).
 Pérdida alta en train y en val, sin mejorar	- Aumentar capacidad del modelo (más neuronas o capas). - Aumentar epochs o reducir Dropout. - Reducir L2 o weight decay.- Bajar learning_rate.
 Pérdida similar en train y val, pero ambas altas	- Usar arquitectura más profunda. - Añadir variables derivadas (lags, volatilidad local). - Verificar estacionariedad de la serie. - Revisar si la serie contiene señal real.
 Pérdida oscilante o errática	- Reducir learning_rate (ej. de 1e-3 → 1e-4). - Usar ReduceLROnPlateau o CosineDecayRestarts. - Revisar escalado de datos.
 Pérdida se estabiliza en un valor alto (sin ruido)	- Probar distintas semillas (random restarts) - Usar optimizadores como AdamW, Nadam o RAdam. - Añadir CosineDecayRestarts. - Reducir batch size (16–32).

Soluciones

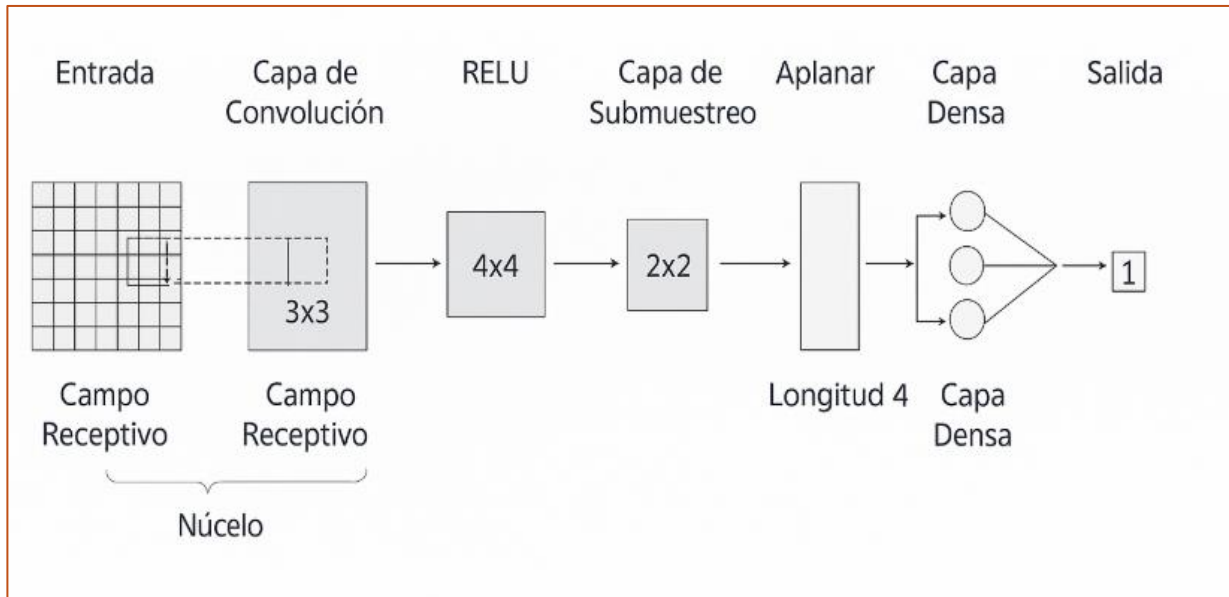
Síntoma observado	Soluciones recomendadas
 Pérdida → NaN o infinita	- Añadir clipnorm=1.0 al optimizador. - Reducir learning_rate. - Normalizar entradas. - Revisar activaciones (ReLU, tanh saturada).
 Pérdida no cambia en absoluto desde el inicio	- Sustituir tanh o sigmoid por ReLU/LeakyReLU. - Usar inicialización He o Glorot.- Aplicar BatchNorm o LayerNorm.
 Mejora inicial y luego empeora rápidamente	- Revisar patience y callback de early stopping. - Aplicar checkpointing. - Ajustar scheduler de LR.
 Pérdida desciende demasiado rápido al inicio y se estanca	- Disminuir learning_rate. - Cambiar inicialización (GlorotUniform) - Añadir ReduceLROnPlateau.
 Curva de validación mejora pero la de entrenamiento no	- Reducir Dropout (ej. 0.5 → 0.2). - Disminuir L2/L1. - Revisar número de unidades.
 Predicciones casi constantes o planas	- Verificar escalado. - Aumentar complejidad. - Revisar activación final (linear).

Redes Convolucionales 1D para Series de Tiempo

Red Neuronal Convolucional CNN

Tipo de red profunda que aprende a **extraer automáticamente patrones locales** (rasgos) de los datos usando la **operación de convolución**.

Fue popularizada en visión por computador, pero funciona igual de bien en **señales 1D** (series de tiempo), **imágenes 2D** y **volúmenes 3D**.



Una CNN apila capas que transforman gradualmente la representación

Un filtro o kernel (matriz pequeña de pesos compartidos) se “desliza” sobre la entrada y calcula productos punto locales:

$$y[i] = \sum_{k=0}^{K-1} w[k] x[i + k]$$

$$Y[i, j] = \sum_{u, v} W[u, v] X[i + u, j + v]$$

- ✓ **Entrada (X):** la señal, imagen o tensor (con canales).
- ✓ **Kernel (W):** pocos parámetros (p. ej., tamaño 3, 5, 7).
- ✓ **Mapa de activaciones (Y):** salida filtrada que resalta un patrón (bordes, picos, transiciones, etc.).

Hiperparámetros de una convolución

Kernel size: longitud/alto del filtro **Padding:** cómo tratas bordes

Stride: paso del deslizamiento

Nº de filtros: cuántos patrones distintos aprendes en esa capa.

Bloques habituales en una CNN



Conv1D/2D/3D → Activación (ReLU/ELU/...)



Aplica varios kernels para producir **mapas de características**. Cada filtro aprende a detectar un. Hiperparámetros: *nº de filtros, kernel size, stride, padding*



Normalización (BatchNorm/LayerNorm) para estabilizar gradientes.



Normaliza activaciones por mini-batch para estabilizar los gradientes y permitir tasas de aprendizaje más altas.



Pooling (Max/Average): reduce resolución preservando rasgos salientes.



Toma una ventana local del mapa de características y la resume, produciendo una salida más pequeña.



Bloques residuales / atajos: ayudan a entrenar redes profundas. (Opcional)



Dropout / Spatial Dropout
Apaga aleatoriamente neuronas/canales durante el entrenamiento para reducir sobreajuste.



Global Average Pooling o Flatten → Capa(s) densa(s) → Salida (regresión o clasificación).



Aplanar (Flatten) o Global Average Pooling. Reduce parámetros antes de la salida.
Capa Densa (Fully Connected). Combina las características aprendidas para tomar la decisión final.

Metodología para Analizar series de tiempo



Padding

La creación de las secuencias a veces requerirán intervención en los bordes de la data por la cantidad irregular

Añade valores ficticios (normalmente ceros) en los bordes de la entrada.

Numpy

Keras -> `layers.Conv1D(..., padding='causal')`

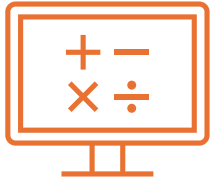
•**Causal**: usa `padding='causal'` (Keras) o rellena **solo a la izquierda** (NumPy/PyTorch) para no “ver el futuro”.

•**Conserva picos**: prefiere `constant=0` o `edge` antes que promedios que suavicen demasiado.

Pooling y normalización

Pooling (submuestreo)

Operación que **resume** un vecindario local para **reducir la resolución** del mapa de características.



MaxPooling1D (k, stride):

$$y_i = \max(x_{i:i+k-1})$$

Average pooling:

$$y_i = \frac{1}{k} \sum_{j=0}^{k-1} x_{i+j}$$

Baja el costo computacional

Introduce **invarianza local** a pequeñas traslaciones

Puede aplanar **shocks** (extremos).

Alternativa: usar **k pequeño (2)**.



Normalización (de activaciones / de datos)

Estabilizar el entrenamiento, acelerar la convergencia y mejorar la generalización.

Batch Normalization (BN):

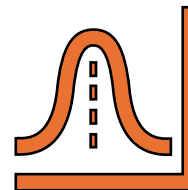
Normaliza cada canal usando media y varianza del mini-batch; luego escala y desplaza aprendibles.

Características:

Entrena más rápido, permite LR mayor.

Con datos muy no-IID o batch pequeño, puede volverse ruidosa.

Suele ir **después de Conv1D y antes de la activación**.



Métricas de comparación

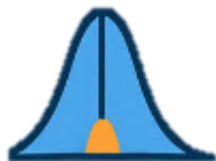
Métricas de precisión puntual

MAE -Error absoluto medio: robusto a outliers moderados; comparable entre datasets si están escalados.

MSE / RMSE: penalizan fuertemente grandes errores; RMSE está en unidades originales.

MAPE / sMAPE: porcentaje (ojo con ceros y valores cercanos a 0) sMAPE corrige sesgos extremos.

Métricas probabilísticas / distribución completa



CRPS
Pinball loss
(Quantile loss)
Winkler score

Diagnóstico de residuales (validez del modelo)



Ljung–Box / Box–Pierce
ARCH-LM
Normalidad

Calibración y cobertura



PICP (Prediction Interval Coverage Probability)
ACE/ECE (Absolute/Expected Calibration Error)
PIT / QQ de residuales pronosticados
Interval Sharpness: ancho medio de PI

Comparación estadística



Diebold–Mariano (DM)
Clark–West
Reality Check / SPA de Hansen

Métricas para funciones de pérdida



Requisitos esenciales

- ✓ Diferenciabilidad
- ✓ Acotada inferiormente y bien condicionada
- ✓ Reducción escalar
- ✓ Estabilidad numérica
- ✓ Vectorizable (sin bucles)

Nombre Keras	Tipo	¿Cuándo usarla?
MeanSquaredError ('mse')	Regresión	Penalizar errores grandes; regresión estándar
RootMeanSquaredError	Regresión	Igual que MSE pero en unidades originales
MeanAbsoluteError ('mae')	Regresión	$y - \hat{y}$
Mean Absolute Percentage Error ('mape')	Regresión	$\frac{y - \hat{y}}{y}$
Mean Squared Logarithmic Error ('msle')	Regresión	Crecimientos relativos, colas derechas
Huber	Regresión robusta	Datos con outliers
LogCosh	Regresión robusta	Suave y robusta
CosineSimilarity	Regresión/embeddings	Alinear direcciones
BinaryCrossentropy	Clasif. binaria	Prob. binaria (sigmoid)
CategoricalCrossentropy	Multiclase	Softmax one-hot
SparseCategoricalCrossentropy	Multiclase (sparse)	Softmax con etiquetas enteras
KLDivergence	Distribuciones	Matching de distribuciones
Poisson	Conteos	Series de conteos
Hinge / SquaredHinge	Máquinas de margen	Clasificación con márgenes